

Energy and Time Complexity for Sorting Algorithms in Java

Osama Kharita, Huzaifa Malbari, Forrest Harlow

Introduction

In today's world technology is a key figure in human life and with that comes advantages and disadvantages, one of the main disadvantages of tech is its growing domination on global energy consumption, as technology currently consumes around 10% of global energy which is a huge amount and it is rapidly increasing with the daily expansion in this sector such as artificial intelligence, cloud computing and the rise of more data centres around the world, this pushes researchers and developers to think about solutions to reduce the consumption amount and one of these methods is improving algorithmic efficiency, however there has always been a debate about whether that actually makes a difference which is exactly what this study came to prove, specifically whether algorithmic efficiency truly reduces energy consumption or not and whether a faster algorithm actually means less energy used, if proven true this would hugely help our world that is already suffering from energy shortages and increasing carbon emissions in some places more than others.

The study focused on four well known sorting algorithms that were chosen specifically because they all have different time complexities giving a wide range to compare, these are Bubble Sort which has a time complexity of $O(n^2)$ making it the slowest of the four, Counting Sort with $O(n+k)$ which makes it very fast when the maximum value in the input is small, and both Merge Sort and Quick Sort which have a time complexity of $O(n \log n)$ making them significantly more efficient than Bubble Sort for large inputs, time complexity is basically a way of describing how an algorithm scales as the input gets bigger, so an algorithm with $O(n^2)$ does dramatically more work than one with $O(n \log n)$ when the input size increases, which in theory should mean it uses more energy too and that is exactly what the study set out to confirm.

Our part as a group was to mimic and remake this study on our own and check if the results would be similar, our group translated the algorithms and pseudocode from the paper into actual working Java code in a suitable environment and then based on this code produced results across a range of input sizes from 25000 to 1000000 elements testing three different input types for each algorithm which are best case, worst case and random case, wall time was measured using the C `gettimeofday` function and energy consumption was measured using Intel's RAPL tool on a Linux system giving us readings in Joules, in almost all of the graphs we produced we found a strong relationship between algorithmic efficiency and energy consumption where fast and efficient algorithms showed minimum energy consumption compared to slow and inefficient ones, which basically confirms the same result the original study came to and with very similar graph measurements and R^2 values above 0.98 across all algorithms confirming the strong correlation between time complexity and real world energy usage.

Energy and Time Complexity of Sorting Algorithms

As we discussed earlier in the introduction section the paper discusses the link between algorithmic time complexity and energy consumption, and what makes this paper stand out from previous work is that while other studies either disagreed on whether time complexity can predict energy or tested on very limited hardware with small input sizes, this paper took a much more thorough and controlled approach to actually settle that debate properly, the researchers tested four algorithms specifically chosen because they all have different time complexities giving a wide and meaningful range to

compare, Bubble Sort with $O(n^2)$ representing the slowest and least efficient, Counting Sort with $O(n+k)$ representing the fastest in ideal conditions, and both Merge Sort and Quick Sort with $O(n \log n)$ sitting in the middle.

The experiments were carried out on Lenovo ThinkPad laptops running Ubuntu Linux in a temperature regulated environment kept at exactly 20°C , input sizes ranged from 25000 to 1000000 elements across three input types per algorithm which are best case, worst case and random case, what is clever about their statistical approach is that they converted the x axis to match the time complexity of each algorithm rather than just plotting against input size, this allowed them to apply linear regression uniformly across all algorithms and calculate R^2 values to measure how strong the correlation is.

They tested this among huge amounts of cases and found a strong link between both, as the graphs clearly show whenever there is a steady increase in time complexity there is a corresponding steady increase in energy consumption, specifically for Counting Sort, Merge Sort and Quick Sort the R^2 values came out above 0.99 and for Bubble Sort above 0.94, this clearly confirms that a faster algorithm will almost certainly use less energy which is exactly what the paper set out to prove, the paper also found something unexpected which is that for Bubble Sort and Merge Sort the random case was actually worse than the theoretical worst case, suggesting that hardware behaviour through branch mispredictions plays a bigger role than theory alone predicts, overall the methodology is solid and the findings are well supported though it is worth noting the study was only carried out on one type of hardware meaning the results may not fully generalise to other environments.

Implementation Details

For each of the four tested sorting algorithms, we followed the pseudocode suggested by the original paper as closely as possible. Though we did end with some differences, most of these were very minor and none compromised our experimental integrity.

```

PROCEDURE BUBBLE_SORT(A)
  n ← length(A)

  FOR i ← 0 TO n - 2
    swapped ← false

    FOR j ← 0 TO n - i - 2
      IF A[j] > A[j + 1] THEN
        SWAP A[j], A[j + 1]
        swapped ← true
      END IF
    END FOR

    IF swapped = false THEN
      BREAK
    END IF
  END FOR
END PROCEDURE

PROCEDURE QUICK_SORT(A, p, r)
  IF p < r THEN
    q ← PARTITION(A, p, r)
    QUICK_SORT(A, p, q - 1)
    QUICK_SORT(A, q + 1, r)
  END IF
END PROCEDURE

PROCEDURE PARTITION(A, p, r)
  pivot ← A[r]
  i ← p - 1

  FOR j ← p TO r - 1
    IF A[j] ≤ pivot THEN
      i ← i + 1
      SWAP A[i], A[j]
    END IF
  END FOR

  SWAP A[i + 1], A[r]
  RETURN i + 1
END PROCEDURE

PROCEDURE COUNTING_SORT(A)
  n ← length(A)
  IF n = 0 THEN
    RETURN A
  END IF

  minVal ← minimum value in A
  maxVal ← maximum value in A
  range ← maxVal - minVal + 1

  count[0..range-1] ← all zeros
  output[0..n-1]

  FOR each value x in A
    count[x - minVal] ← count[x - minVal] + 1
  END FOR

  FOR i ← 1 TO range - 1
    count[i] ← count[i] + count[i - 1]
  END FOR

  FOR i ← n - 1 DOWN TO 0
    x ← A[i]
    idx ← x - minVal
    output[count[idx] - 1] ← x
    count[idx] ← count[idx] - 1
  END FOR

  FOR i ← 0 TO n - 1
    A[i] ← output[i]
  END FOR

  RETURN A
END PROCEDURE

PROCEDURE MERGE_SORT(A, left, right)
  IF left ≥ right THEN
    RETURN
  END IF

  mid ← floor((left + right) / 2)

  MERGE_SORT(A, left, mid)
  MERGE_SORT(A, mid + 1, right)

  MERGE(A, left, mid, right)
END PROCEDURE

PROCEDURE MERGE(A, left, mid, right)
  n1 ← mid - left + 1
  n2 ← right - mid

  L[0..n1-1], R[0..n2-1]

  FOR i ← 0 TO n1 - 1
    L[i] ← A[left + i]
  END FOR

  FOR j ← 0 TO n2 - 1
    R[j] ← A[mid + 1 + j]
  END FOR

  i ← 0, j ← 0, k ← left

  WHILE i < n1 AND j < n2
    IF L[i] ≤ R[j] THEN
      A[k] ← L[i]
      i ← i + 1
    ELSE
      A[k] ← R[j]
      j ← j + 1
    END IF
    k ← k + 1
  END WHILE

  WHILE i < n1
    A[k] ← L[i]
    i ← i + 1
    k ← k + 1
  END WHILE

  WHILE j < n2
    A[k] ← R[j]
    j ← j + 1
    k ← k + 1
  END WHILE
END PROCEDURE

```

Note that we have written these solely with primitive integers in mind, as that is the only data type tested by the original paper. All of our tests build upon these with a custom harness for Intel’s RAPL written in C. We used a recent ASUS TUF Gaming laptop to run the tests, and were unfortunately not able to find another compatible machine. However, the results of our experiments should still generalize to other systems.

Experiments

The same RAPL tool (written in C) as the one in the paper was used to measure energy usage and wall time. The only modification was to make the tool itself run each test 10 times and output the average, instead of printing all 10 cases separately. The tool measures the energy and time by executing the java program using the system() function and recording the time and energy usage for the duration that the java command ran.

Bash scripts were used to run the RAPL tool with each algorithm along with its worst, best and random cases arrays of different sizes (as specified in the paper). The results were output to 4 csv files named after the algorithms. Finally, the data was plotted using Google sheets. The final results and graphs are in the Graphs folder.

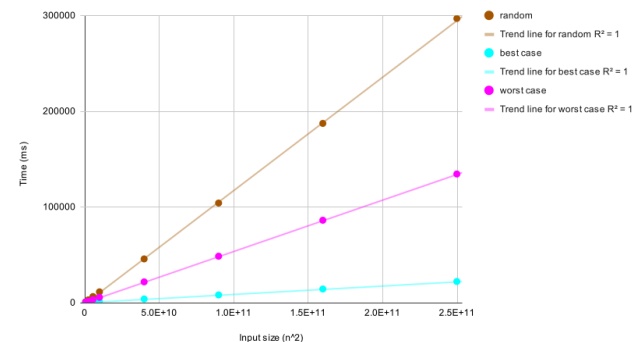
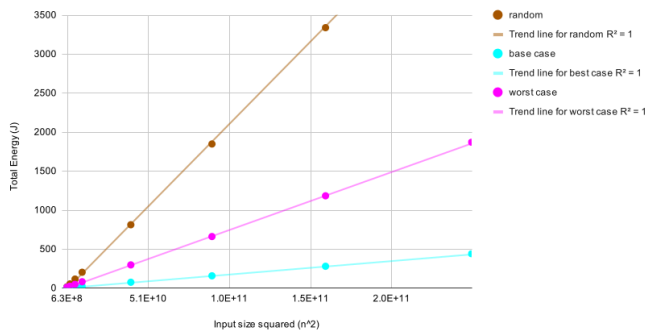
Results

The results across all four sorting algorithms were very strong and closely matched what the original paper found, in almost all of the graphs produced the R^2 value came out above 0.98 which confirms a near perfect linear relationship between time complexity and both energy consumption and wall time,

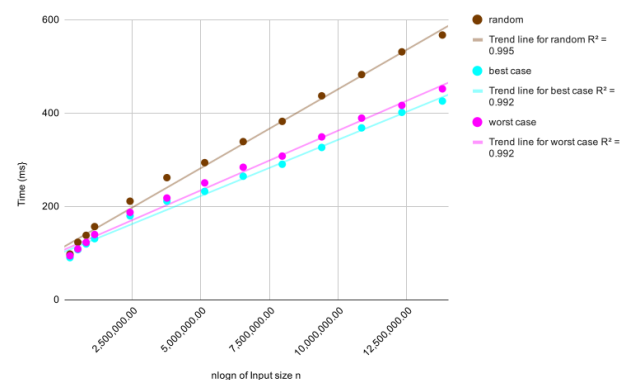
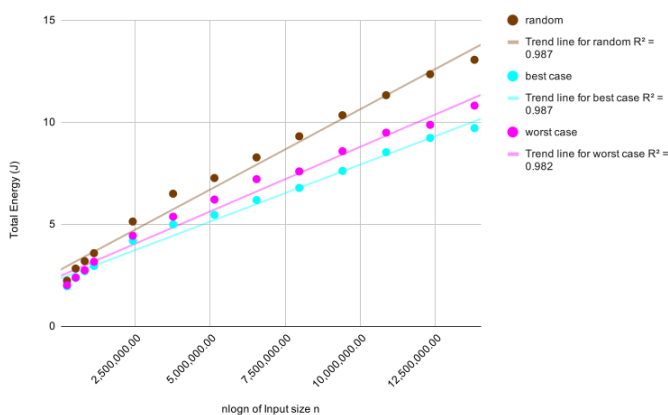
this basically approves the paper's central claim and our own finding that there is a strong link between energy consumption and algorithmic efficiency, kind of like a linear relationship where if one increases or decreases the other one corresponds accordingly, for example looking at QuickSort best case the R^2 value for energy was 0.99 and for wall time was 0.998 which is almost perfectly linear, and similarly for MergeSort the results showed the same strong upward trend across all input sizes with R^2 values consistently above 0.99, this pattern was visible across all algorithms regardless of input type which is a strong indicator that time complexity is a reliable predictor of real world performance and energy usage.

However there was one interesting finding as well as what we discussed earlier which is that for MergeSort and BubbleSort the theoretical worst case input did not actually produce the worst real world performance, instead the random case was worse, specifically our MergeSort results showed that the random case consumed more energy and took more time than the worst case input which used alternating elements, this directly replicates the paper's surprising secondary finding and is clearly visible when comparing the three MergeSort graphs side by side, the paper suggests this happens because of something called branch mispredictions in modern CPUs, basically when the input is reverse sorted or alternating the CPU can learn the pattern and predict the next instruction correctly, but when the input is completely random the CPU keeps guessing wrong and has to correct itself repeatedly which wastes CPU cycles and therefore uses more energy.

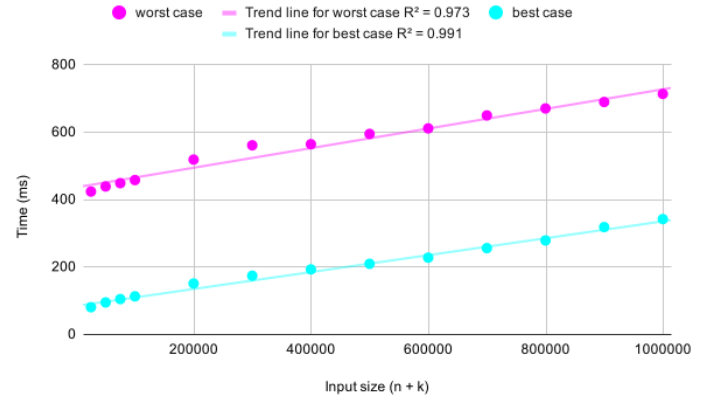
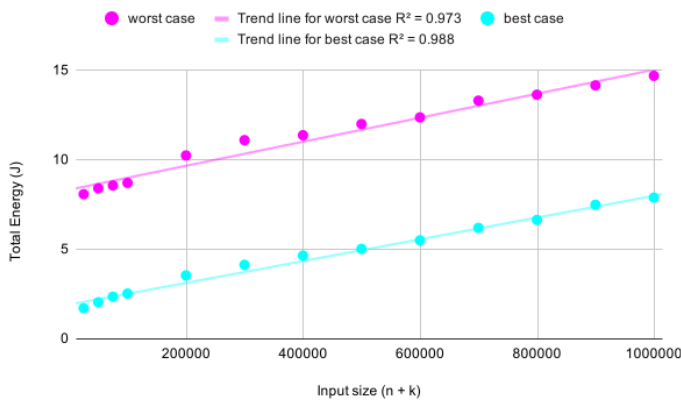
Overall we can clearly see that our results align very closely with the paper's findings, the strong R^2 values across all algorithms and all input types confirm that time complexity is a reliable predictor of energy consumption in practice, and the branch misprediction finding adds an interesting hardware level explanation for why real world results do not always perfectly match theoretical predictions.



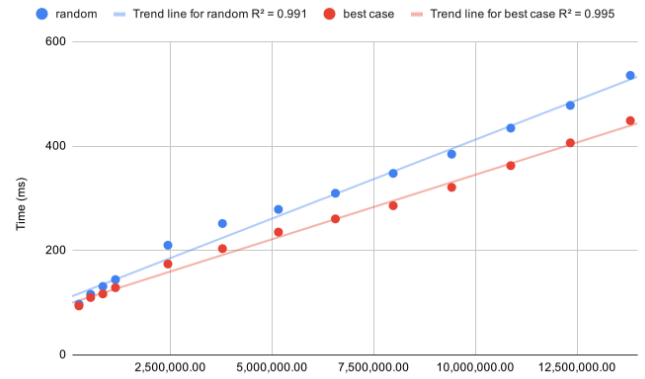
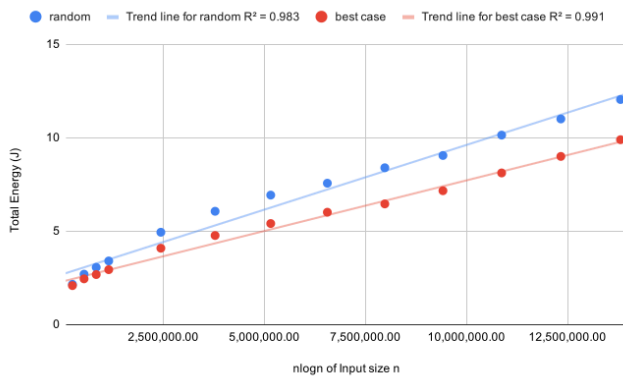
Bubblesort Energy, Time



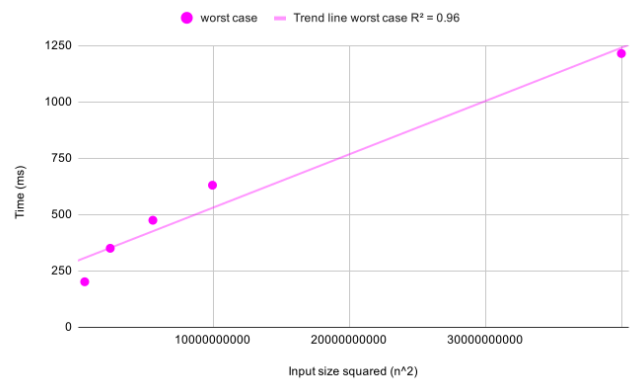
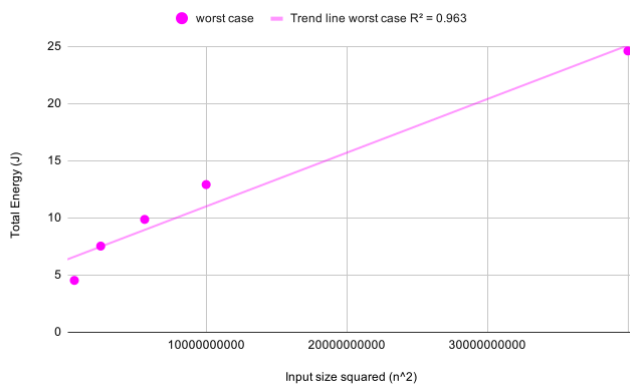
MergeSort Energy, Time



CountingSort Energy, Time



QuickSort Random and Best Energy, Time



QuickSort Worst Energy, Time

Discussion

The results have basically ended the long standing debate among academics on whether algorithmic efficiency truly has a direct effect on energy consumption, both our results and the paper's results are

close enough to nearly identical confirming that if an algorithm is efficient enough it reduces energy consumption in direct accordance to the time it takes to execute its job, this means software developers do not need to directly measure energy consumption every time they write code, they can simply look at the time complexity of their algorithm and use that as a reliable guide for energy efficiency which massively simplifies writing energy efficient software in practice. Where our results differed slightly from the paper was in the exact numbers rather than the overall pattern, this is expected given that the paper used specific hardware in a temperature controlled environment while our setup was different, however the R^2 values above 0.98 across nearly all cases confirm that the relationship holds regardless of the specific hardware used which actually strengthens the paper's argument further.

However what was discovered newly and somewhat shockingly was that modern CPUs might actually perform better in worst case scenarios than in random cases, the reason behind this is that modern CPUs use branch prediction where the processor tries to predict the next instruction before it arrives, in worst case scenarios like reverse sorted arrays the pattern is very predictable so the CPU handles it smoothly, but in random cases the CPU keeps guessing wrong and wastes cycles correcting itself, these wrong guesses are called branch mispredictions.

Conclusion

Overall our results confirmed the paper's central finding that time complexity is a reliable predictor of energy consumption with R^2 values being above 0.98 across nearly all tested cases, meaning faster algorithms genuinely use less energy in practice which is a valuable and important result for software developers and companies looking to reduce energy consumption worldwide, the secondary finding about branch mispredictions was equally interesting showing that modern CPUs can actually struggle more with random inputs than theoretical worst case inputs which is something worth investigating more about it, In terms of limitations our study was carried out on specific hardware meaning results may vary across different machines and future work should verify these findings across different environments, but overall the project successfully replicated the paper's key findings.